

Reconfigurable FIR Filter

수 강 과 목 : 디지털 시스템 설계
대 학 : IT대학
학 과 : 컴퓨터학부, 전자정보공학부
조 : 6
조 원 학 번 : 20212985 20231566 20232065
조 원 이 름 : 이정우(팀장), 박도유, 이다정
작 성 일 자 : 2025.12.9.(목)

목 차

목 차	i
I. 프로젝트 개요 및 팀 구성	1
1.1 프로젝트 목표	1
1.2 팀 구성원 및 역할 분담	1
1.3 설계 순서	1
II. 구현 구조	2
2.1 Top Module Architecture	3
2.2 Delay Chain	3
2.3 Controller (FSM)	4
2.4 SpSram Interface	5
2.5 MAC Unit	6
2.6 Final Summation	6
III. 검증 및 결과	7
3.1 Test bench	7
3.2 Simulation 결과	8
IV. 고찰	9
4.1 구조 개선안	10
4.2 느낀 점	10

I. 프로젝트 개요 및 팀 구성

1.1 프로젝트 목표

본 프로젝트는 디지털 통신 시스템에서 핵심적인 역할을 하는 FIR(Finite Impulse Response) Filter를 FPGA 상에서 동작 가능한 하드웨어로 설계하는 것을 목표로 합니다.

핵심 목표:

- 1) 고속 처리: 600kHz Sampling Rate 및 200kHz Symbol Rate를 지원하며, 4개의 병렬 MAC 구조로 Sample 처리 제한 시간(20clock) 내에 33-Tap 연산을 완료한다.
- 2) 가변성(Reconfigurability): 필터 계수를 ROM이 아닌 SpSram에 저장하여, 하드웨어 변경 없이 필터 특성을 실시간으로 업데이트할 수 있도록 한다.
- 3) 병렬 구조 최적화: 4개의 MAC Unit과 4-Bank SRAM을 활용한 병렬 처리 구조를 통해 연산 효율을 극대화한다.

1.2 팀 구성원 및 역할 분담

이정우 (팀장): Controller(FSM), SpSram Interface 설계, Top Module 통합

박도유 : Datapath(Delay Chain, MAC Unit) 설계

이다정 : Verification (TestBench & Simulation), 보고서 작성 및 발표

1.3 설계 순서

(11/26 ~ 12/2): 설계 구조 설정, Controller와 Datapath를 분리하여 RTL 코딩

(12/3 ~ 12/4): 서브 모듈 코딩

(12/5 ~ 12/6): 단위 모듈(MAC, FSM, RAM)의 기능 검증 후 Top Module 통합

(12/7 ~ 12/8): Testbench 작성 후 검증

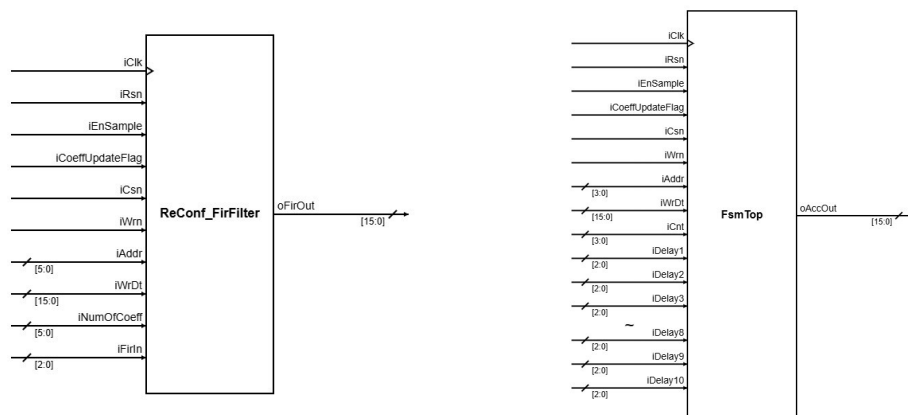
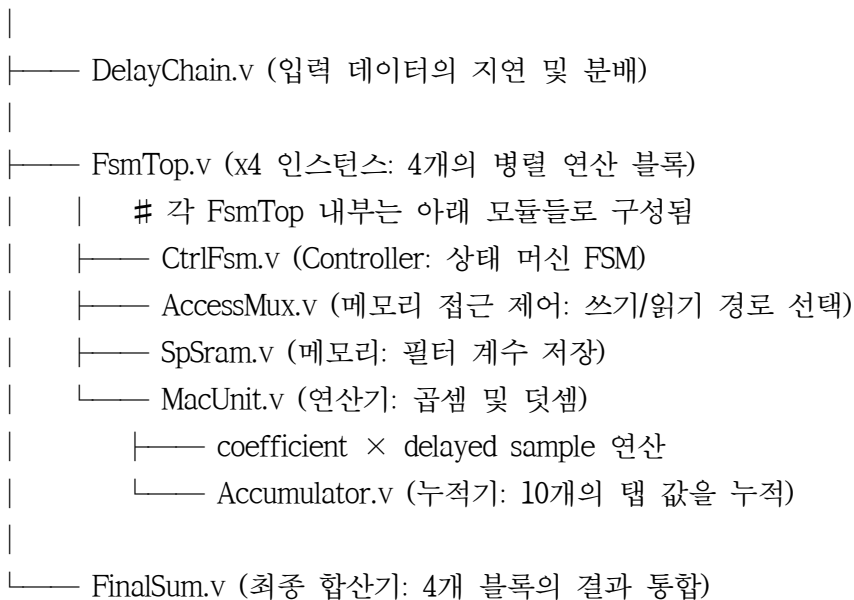
(12/8 ~ 12/9): 구조 개선안 분석, 보고서 작성

II. 구현 구조

본 장에서는 Reconfigurable FIR Filter가 어떤 구조로 동작하는지에 대해 Top Module 기준으로 각 서브 블록의 역할과 내부 동작을 기술한다. 전체 FIR 필터는 입력 샘플(iFirIn)을 Delay Chain을 통해 40-tap 지연값으로 확장한 뒤, 4개의 병렬 MAC Unit이 각각 10개의 탭을 계산하고, 최종적으로 Final Summation 모듈이 4개의 부분합(partial sum)을 더하여 FIR 출력 oFirOut을 생성하는 구조로 구현하였다. 또한 계수 업데이트(coefficient update) 기능을 지원하기 위해 각 MAC Unit은 독립적인 10-depth SpSram을 포함한다.

■ 전체 모듈 구조도

ReConf_FirFilter.v (최상위 모듈: Top Module)



2.1 Top Module Architecture

목표 사양인 33-Tap 연산을 단일 MAC으로 순차 처리할 경우 최소 33 클럭 이상이 소요되어, 제한 시간(20 클럭)을 초과한다. 이를 해결하기 위해 전체 탭을 4개의 병렬 블록(FsmTop)으로 분할하였다. 각 블록은 최대 10개의 탭(총 40-Tap 용량)을 담당하며, 병렬 처리를 통해 20 클럭 내에 충분한 Timing Margin을 확보할 수 있다.

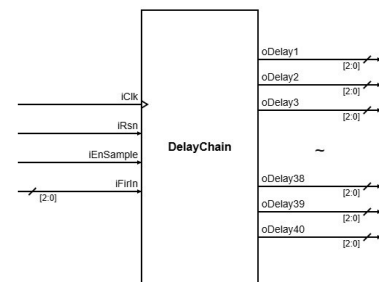
하드웨어 변경 없이 필터 특성을 수정하기 위해 iCoeffUpdateFlag 신호를 모니터링한다. 동작 모드(Operation)와 업데이트 모드(Update)를 실시간으로 전환하여, 필터 계수를 유동적으로 변경할 수 있는 구조를 갖추었다.

■ Top-Level 동작 흐름

- 1) 입력 샘플 수신 후 DelayChain에서 40단 지연 생성
- 2) iNumOfCoeff(33)에 따라 4개의 MAC Unit 중 사용 개수 자동 조정
(예: 17 tap → MAC1:10, MAC2:7)
- 3) 각 MAC Unit 내부 SRAM에서 계수를 읽고 Delay tap과 곱한 뒤 누산
- 4) 각 MAC Unit의 부분합 4개를 FinalSum 모듈이 더함
- 5) 오버플로우 방지용 Saturation 적용 후 oFirOut 출력

2.2 Delay Chain

Delay Chain은 입력된 샘플 데이터를 순차적으로 지연시켜 FIR 필터 연산에 필요한 탭 데이터(Tap Data)를 생성하고, 이를 연산 유닛에 공급하는 모듈이다.



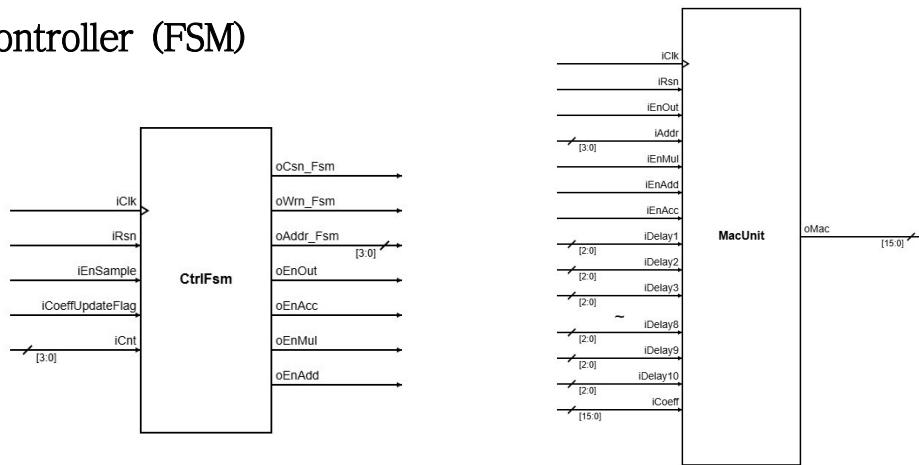
■ 동작 원리

본 모듈은 40단으로 이루어진 Shift Register로 구성되어 있다. 3비트 입력 신호(iFirIn)가 들어오면, 매 샘플링 클럭(iEnSample=1)마다 저장된 데이터를 한 칸씩 이동(Shift)시킨다. 이를 통해 현재 시점부터 과거 40번째 시점까지의 샘플 데이터를 실시간으로 유지하며, 필터 연산에 즉시 사용할 수 있도록 준비한다.

■ 병렬 데이터 분배

생성된 40개의 탭 데이터는 4개의 병렬 MAC Unit이 동시에 연산을 수행할 수 있도록 분산 공급된다. 전체 40개 탭을 10개씩 4개의 그룹으로 나누어 각 MAC Unit (예: MAC1은 1~10 tap, MAC2는 11~20 tap)에 전용 경로로 전달한다. 이러한 병렬 배선 구조는 다수의 연산 유닛이 데이터 로딩 대기 시간 없이 동시에 연산을 시작할 수 있게 하여 처리 효율을 극대화한다.

2.3 Controller (FSM)



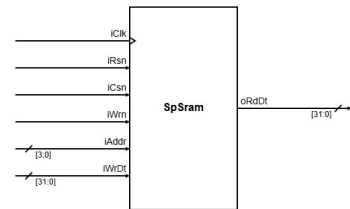
각 MAC Unit(FsmTop 내부)의 핵심 제어부는 Controller(FSM)으로, SRAM 읽기/쓰기를 관리하고, 곱셈·덧셈 타이밍을 제어한다. 즉, 메모리 접근과 연산 흐름을 정밀하게 제어한다.

상태	기능
Idle	샘플 대기 / 업데이트 대기
Update	coefficient write 지속
MemRd	SRAM read, 주소 증가, 곱셈 시작
Wait / Wait2	곱셈 pipeline 2clk 보정
Out	Acc 결과 출력

SRAM의 읽기 지연(1cycle)과 곱셈기의 계산 지연(1cycle)을 보정하기 위해, 제어 신호(wEnMul, wEnAcc)에 Shift Register 기반의 지연 보상 로직을 적용하였다. 즉, 데이터가 도착하는 정확한 시점에 누적 연산이 수행되도록 누산기 컨트롤 신호는 2cycle 지연된 pipeline 구조로 생성하였다.

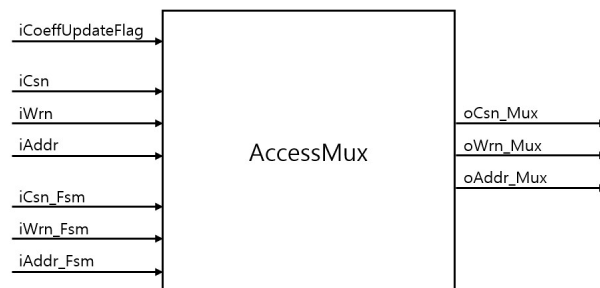
2.4 SpSram Interface

기존의 고정된 ROM 방식 대신, Read/Write가 가능한 SpSram(Single-Port Static RAM)을 적용하여 필터 계수의 실시간 업데이트를 구현하였다. SpSram 인터페이스는 동작 모드에 따른 접근 제어와 병렬 연산을 위한 대역폭 확보를 핵심 기능으로 한다.



1) 동작 모드에 따른 접근 제어 (Access Mux)

- Update Mode ($iCoeffUpdateFlag = 1$): 외부 입력 신호($iAddr$, $iWrDt$, $iWrn$)가 SpSram의 포트에 연결되어, 새로운 필터 계수를 메모리에 Write 한다. 이를 통해 하드웨어 재합성 없이 필터 특성을 변경할 수 있다.
- Filtering Mode ($iCoeffUpdateFlag = 0$): FSM 내부에서 생성된 제어 신호($iAddr_Fsm$, $iWrn_Fsm$)가 연결된다. 이때 $iWrn_Fsm$ 은 Read 모드(High)로 고정되며, 매 클럭마다 $wAddr_Fsm$ 이 증가하며 연산에 필요한 계수를 고속으로 인출하여 MAC Unit에 공급한다.



2) 병렬 처리를 위한 4-Bank 구조

4개의 MAC Unit이 동시에 연산을 수행해야 하므로, 단일 메모리 구조에서는 병목 현상이 발생한다. 이를 해결하기 위해 총 40개의 계수 공간을 4개의 독립적인 Memory Bank로 분할 배치하였다. 각 MAC Unit은 자신에게 할당된 전용 SpSram Bank에 독립적으로 접근하므로, 4개의 병렬 MAC 연산이 충돌 없이 동시에 수행된다.

2.5 MAC Unit

MAC Unit은 필터의 핵심 연산인 곱셈과 누적을 수행하는 모듈이다. 본 설계에서는 고속 동작 환경에서의 타이밍 제약을 충족하고 Critical Path를 최소화하기 위해 Multiplier(곱셈) → Pipeline Register(중간 저장) → Accumulator(누적)의 3단 파이프라인 구조를 적용하였다. 각 MAC Unit은 할당된 10개의 Tap에 대한 부분합을 계산한다.

(1) 데이터 선택 및 파이프라인 곱셈

FSM의 제어 신호(iAddr)에 따라 Delay Chain에서 전달된 10개의 탭 데이터 중 하나와, SpSram에서 읽어낸 계수(h[k])가 동시에 선택된다. 이 두 값은 곱셈기에서 연산된 후, 결과값이 즉시 누산기로 가지 않고 파이프라인 레지스터(rMultResult)에 저장되어 이후 누산 과정에서 사용된다.

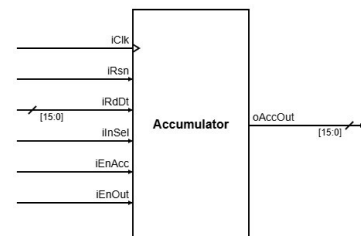
(2) 누산 연산

Accumulator 모듈은 파이프라인 레지스터에 저장된 곱셈 결과를 순차적으로 더해 MAC Unit의 부분합을 생성한다. FSM은 다음 제어 신호로 누산기를 동작시킨다.

iInSel : 첫 탭인지 여부(0 → Acc 초기화, 1 → 누적)

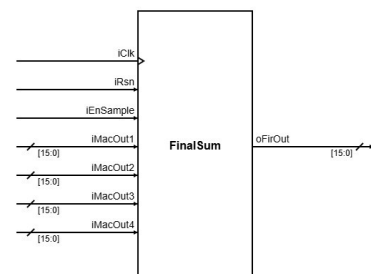
iEnAcc : 새로운 누적 결과 latch

iEnOut : 10개 연산 완료 시 결과 출력



2.6 Final Summation

FinalSum 모듈은 4개의 병렬 MAC Unit에서 분산 처리된 부분합들을 취합하여, 최종적인 FIR 필터 출력을 생성하고 데이터 비트 폭을 조절한다.



1) 부분합 통합 및 비트 확장

4개의 MAC Unit으로부터 입력받은 부분합(iMacOut1 ~ iMacOut4)을 모두 더해 전체 40-Tap 필터의 연산 결과를 도출한다. 이때 16비트 데이터 4개를 합산하는 과정에서 값이 커질 수 있으므로, 내부 연산 변수는 20비트로 확장하여 오버플로우 없는 정밀한 덧셈을 수행한다.

2) Saturation 로직 적용 (Overflow Protection)

내부 연산 결과는 20비트이지만 최종 출력(oFirOut)은 16비트로 제한되어야 한다. 단순 절삭 시 발생할 수 있는 부호 역전 현상을 방지하기 위해 Saturation 로직을 적용하였다.

- Positive Overflow: 양수 최대값을 초과할 경우 +32767(0x7FFF)로 고정한다.
- Negative Overflow: 음수 최소값보다 작을 경우 -32768(0x8000)로 고정한다.

이 과정을 통해 출력 오차를 최소화하고 FPGA 연산의 안정성을 확보하였으며, 최종 결과는 샘플링 타이밍(iEnSample=1)에 맞춰 갱신된다.

Ⅲ. 검증 및 결과

3.1 Test bench

본 설계의 기능 검증을 위해 Verilog HDL 기반의 Testbench(tb_ReConf_FirFilter)를 작성하여 전체 FIR Filter의 동작을 시뮬레이션하였다. Testbench는 다음의 4가지 주요 역할을 수행한다.

(1) Clock / Sample Enable / Reset 생성

- Clock rate: 12MHz → 83.33 ns 주기의 iClk 신호 생성
- Sampling rate: 600kHz → Clock rate를 20분주하여 iEnSample 신호 생성
- Reset(iRsn) 시뮬레이션 초기 200 ns 동안 비활성 상태(0)를 유지한 후 1로 전환

(2) Impulse 입력 생성

iEnSample이 High일 때마다 카운트 (cnt64)

64번째 sample마다 (cnt64==63) iFirIn에 1(3'b001)을 입력

나머지 샘플은 모두 0(3'b000) 입력

(3) Coefficient Update 기능 검증

본 프로젝트의 핵심 목표인 '실시간 계수 업데이트' 기능을 검증하기 위해, 다음과 같은 순차적 시나리오를 수행한다.

Step 1 (Update Mode): iCoeffUpdateFlag를 1로 설정하여 업데이트 모드 진입

Step 2 (Write): 주소 0~32번까지 필터 계수 데이터를 순차적으로 SRAM에 Write

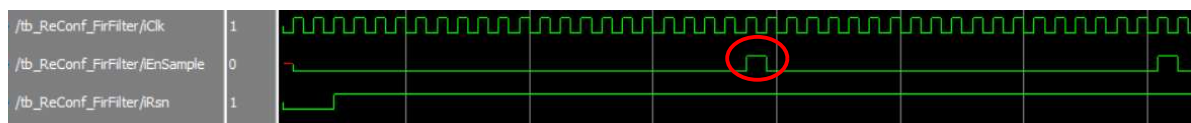
Step 3 (Run Mode): 쓰기 완료 후 iCoeffUpdateFlag를 0으로 변경하여 필터링 동작

(4) Output Valid 제어 및 모니터링

FIR 필터의 연산 지연(Latency)을 고려하여 출력 데이터의 유효성을 판단한다. 샘플 활성화 신호(iEnSample)를 1 클럭 지연시킨 rOutputValid 신호를 생성하고, 이 신호가 High인 구간의 oFirOut 값을 유효한 필터 출력으로 간주하여 검증한다.

3.2 Simulation 결과

Simulation은 ModelSim 환경에서 수행하였으며, Waveform을 기반으로 결과를 분석하였다.



(1) Clock / Reset / Enable 동작 정상

- iClk은 일정 주기로 토글되며 정상 생성됨
- Reset(iRsn)이 1이 되는 시점 이후 DUT 내부 상태가 정상 활성화됨
- iEnSample이 정확히 20clk마다 High로 발생

이는 샘플 기반 FIR 동작의 기본 전제 조건이 충족됨을 의미한다.

4.1 구조 개선안

1) FSM 단일화

현재 MAC Unit마다 독립된 FSM을 사용하고 있으나, 이는 제어 로직의 중복과 리소스 증가를 초래한다. 이를 중앙 집중식 FSM 한 개로 통합하여 각 MAC Unit에는 최소한의 제어 신호만 전달하도록 구조를 단순화할 수 있다. 이 방식은 전체 제어 흐름을 명확하게 하고 타이밍 관리 및 디버깅을 용이하게 한다.

2) FSM Wait 단계 축소

SRAM Read 지연(1 cycle)과 곱셈기 Pipeline(1 cycle)을 모두 FSM 상태로 처리하면서 Wait 단계가 2단으로 늘어난 상황이다. 이를 SRAM Read 완료를 확인하는 최소한의 Wait 단계만 유지하고, 곱셈기 지연 보정은 내부 Pipeline 레지스터로 처리하도록 설계를 정리할 수 있다. 이를 통해 FSM 상태 수가 감소하고 동작 타이밍이 명확해진다.

3) 계수 대칭성 기반의 메모리 최적화

FIR 필터 계수가 대칭 구조를 가질 경우, 전체 계수를 저장할 필요 없이 절반만 저장하고 $(x[n-k] + x[n-(N-1-k)]) * h[k]$ 형태로 연산할 수 있다. 이를 통해 SRAM 사용량을 약 50% 줄이고 메모리 접근 횟수를 감소시켜 전체 연산 효율을 향상시킨다.

4.2 느낀 점

알고리즘적으로는 단순한 FIR Filter도, 실제 FPGA에서 Timing Constraint를 지키며 파이프라인을 구성하는 것은 훨씬 복잡함을 체감하였다. 또, 33 Tap을 20 Clock 내에 연산하기 위해서는 단순히 MAC 속도를 올리는 것이 아니라, Delay, SRAM, FSM, MAC 간의 데이터 흐름을 정교하게 설계해야 한다는 점을 실감했다.

전체적으로 FIR 알고리즘의 이해에서 시작해, 실제 하드웨어 설계 · 코딩 · 검증까지 이어지는 전 과정의 흐름을 모두 경험한 의미 있는 작업이었다. 팀 프로젝트를 통해 구성원 간 역할을 분담하고 협업하는 과정 역시 큰 도움이 되었다.